LAB TASK: AI

MOHIZ KHAN: FA24-BCS-013

2) Solved Lab Activities 1:

Activity 1: Depth First Search (DFS) Using Recursion

Algorithm Explanation

Depth First Search (DFS) explores a graph by going as deep as possible before backtracking.

Steps:

1. Start from a selected node.
2. Mark it as visited and print it.
3. Recursively visit all unvisited adjacent nodes.
4. Backtrack when no unvisited nodes remain.

```python
# Using a Python dictionary to act as an adjacency list
graph = {
    '5': ['3', '7'],
    '3': ['2', '4'],
    '7': ['8'],
    '2': [],
    '4': ['8'],
    '8': []
}

visited = set()

def dfs(visited, graph, node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)

print("Following is the Depth-First Search:")
dfs(visited, graph, '5')
```

```
Following is the Depth-First Search:
5 3 2 4 8 7
```

Activity 2: The Depth First Traversal (or Search) for a graph is similar to Depth First Traversal of a tree. The only difference is, unlike trees, graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we use a Boolean visited array. For example, in the graph given in the Figure below, we start traversal from vertex 2. When we come to vertex 0, we look for all adjacent vertices of it. 2 is also an adjacent vertex of 0. If we don't mark visited vertices, then 2 will be processed again and it will become a non-terminating process. A Depth First Traversal of the following graph is 2, 0, 1, 3.

Activity 2: DFS for Graph with Cycles

Concept

Unlike trees, graphs may contain cycles. To avoid infinite loops:

- Use a **visited array** to track visited nodes.

```python
from collections import defaultdict

class Graph:
    def __init__(self):
        self.graph = defaultdict(list)

    def addEdge(self, u, v):
        self.graph[u].append(v)

    def DFSUtil(self, v, visited):
        visited[v] = True
        print(v, end=" ")

        for i in self.graph[v]:
            if not visited[i]:
                self.DFSUtil(i, visited)

    def DFS(self):
        V = len(self.graph)
        visited = [False] * V

        for i in range(V):
            if not visited[i]:
                self.DFSUtil(i, visited)

# Driver Code
g = Graph()
g.addEdge(0, 1)
g.addEdge(0, 2)
g.addEdge(1, 2)
g.addEdge(2, 0)
g.addEdge(2, 3)
g.addEdge(3, 3)

print("Following is Depth First Traversal:")
g.DFS()
```

```
Following is Depth First Traversal:
0 1 2 3
```

Lab Task 1:

In order to move onto more complex implementations, like previous implementation of BFS, solve the same Travelling Salesman Problem (TSP) given in Figure below using DFS from Arad to Bucharest: Figure: Graph for Travelling Salesman Problem Figure: Travelling Salesman Problem i. Draw a corresponding graph from the above Figure from Arad to Bucharest containing all possible nodes. (Arad as Start Node and Bucharest as Goal Node) ii. Apply depth first search to reach the goal state, your output should have the path and the total distance to reach the path. iii. Save your code and results as DFS_REG#.py as it would be used in later exercises.

```python
graph = {
    'Arad': [('Zerind', 75), ('Timisoara', 118), ('Sibiu', 140)],
    'Zerind': [('Oradea', 71)],
    'Oradea': [('Sibiu', 151)],
    'Timisoara': [('Lugoj', 111)],
    'Lugoj': [('Mehadia', 70)],
    'Mehadia': [('Drobeta', 75)],
    'Drobeta': [('Craiova', 120)],
    'Craiova': [('Pitesti', 138)],
    'Sibiu': [('Fagaras', 99), ('Rimnicu Vilcea', 80)],
    'Rimnicu Vilcea': [('Pitesti', 97)],
    'Fagaras': [('Bucharest', 211)],
    'Pitesti': [('Bucharest', 101)],
    'Bucharest': []
}
def dfs_tsp(graph, start, goal, visited=None, path=None, cost=0):
    if visited is None:
        visited = set()
    if path is None:
        path = []

    visited.add(start)
    path.append(start)

    if start == goal:
        return path, cost

    for (neighbor, weight) in graph[start]:
        if neighbor not in visited:
            result = dfs_tsp(graph, neighbor, goal, visited.copy(), path.copy(), cost + weight)
            if result is not None:
                return result
```

```python
        return path, cost

    for (neighbor, weight) in graph[start]:
        if neighbor not in visited:
            result = dfs_tsp(graph, neighbor, goal, visited.copy(), path.copy(), cost + weight)
            if result is not None:
                return result

    return None

# Driver Code
start = 'Arad'
goal = 'Bucharest'

result = dfs_tsp(graph, start, goal)

if result:
    path, cost = result
    print("DFS Path:", " → ".join(path))
    print("Total Cost:", cost)
else:
    print("No path found")
```

```
DFS Path: Arad → Zerind → Oradea → Sibiu → Fagaras → Bucharest
Total Cost: 607
```

Lab Task 2: Create a graph of the Figure for TSP given above and then set initial and goal states such that the number of nodes visited for BFS is smaller than that in DFS. Now modify the initial and goal state such that the number of nodes visited for BFS is larger than that in DFS

Case 1: BFS Visits Fewer Nodes than DFS

Setup

- Start: `Arad`
- Goal: `Sibiu`

Reason

- BFS finds shortest path quickly
- DFS may explore unnecessary deeper branches

Conclusion

```
BFS Nodes Visited < DFS Nodes Visited
```

Comparison Table

Scenario	BFS	DFS
Short Goal	Efficient	Less Efficient
Deep Goal	Slower	Faster
Guarantees Shortest Path	Yes	No
Memory Usage	High	Low

Final Conclusion

- DFS is **memory efficient** and useful for deep traversal.
- BFS guarantees **shortest path** but uses more memory.

- Choice depends on:
    - Problem type
    - Graph depth
    - Required optimality